

[文献资料](#)[下一篇：团队合作与重量共享](#)

向量数据

目录

1. 向量数据

○ 概览

- 高级关系数据
- Example1：以下示例演示 offsets 的使用。它类似于 Expression mode 中 `rel_ret_cust` 的实现。
- 示例 2：以下示例等同于表达式 `sum (log (rel_ret_comp), 21)`
- 示例3：以下 alpha 以不同方式计算公司竞争对手的月度收益。cap 的月度收益比 close 是更好的估计，因为它对股票拆分具有稳健性。该 alpha 的 Simulation 速度比上面的快得多，因为它没有额外的 for 循环。

向量数据

概览

并非所有数据集都可以与 Expressions 一起使用。有些数据集非常丰富，不适合在 Expression 模式下使用，因为它们只能以向量格式表示，因此必须在 Python 模式下使用。

使用 Vector Data 时需要记住的一些属性：

- 这些字段只能在使用 Python 构建 alphas 时使用。
- 这些字段采用 Offset/Vector 形式，而不是 Matrix。
- 请注意，所有 *_offset 字段都通过 `[di][ii]` 访问，并返回一个元组 (start, num)，其中 start 是向量数据中的起始位置，而 num 是条目数量。
- 非 offset 字段通过 `<field_name>[off]` 访问，其中 off 应位于 start 和 (start + num) 之间。对于 *_id 字段，标识符指的是 `alpha.shape[0]` 中的 ii。

WebSim 中可用以下向量数据集：

除了可用于 expression alphas 的基本 Relationship 数据外，WebSim 现在还支持这些用于 Python Alphas 的 Advanced Relationship 数据名称。

该数据适用于 Customers（cust）、Competitors（comp）、Strategic Partners（part）及其 Relationships（rel）类别。

数据名称	描述	范围	Python 用法
rlv_cust_offset	客户相关字段的偏移量（int[2...	NA（检查是否 >=0）	博士。GetData（"rlv_cust_offset"）
rlv_cust_id	客户标识符（int）	ii 位于区间 [0， alpha size） 内	博士。GetData（"rlv_cust_id"...
rlv_cust_rev	来自客户的收入占比（float）	0-100	博士。GetData（"rlv_cust_rev"）
rlv_comp_offset	竞争对手相关字段的偏移量（int[2]	匿名	博士。GetData（"rlv_comp_offset"）
rlv_comp_id	竞争对手标识符（int）	ii 位于区间 [0， alpha size） 内	相关公司 ID
rlv_rel_offset	关系相关字段的偏移量（int[2...	匿名	博士。GetData（"rlv_rel_offset"）
rlv_rel_id	关系标识符（int）	ii 位于区间 [0， alpha size） 内	博士。GetData（"rlv_rel_id"）
rlv_rel_overlap	两家公司共同参与的行业数量（int）	通常为 0-100	博士。GetData（"rlv_rel_overlap"）
rlv_rel_focus	两家公司共同专注的行业数量（int）	通常为 0-100	博士。GetData（"rlv_rel_focus"）
rlv_part_offset	合作伙伴相关字段的偏移量（int[2]	匿名	博士。GetData（"rlv_part_offset"）
rlv_part_id	合作伙伴标识符（int）	ii 位于区间 [0， alpha size） 内	博士。GetData（"rlv_part_id"...
rlv_part_type	公司与其合作伙伴之间的关系类型（int）	范围：0-13。整数值对应以下关系类型：DISTRIB、EINVEST、LICENIN、MANUFAC、MARKTNG 等	博士。GetData（"rlv_part_type"）

Delay 1 Alpha 示例：

在以下所有示例中，start 是向量数据中的起始位置，而 num 是条目的数量。

Example 1: 以下示例等同于表达式 `sum(log(rel_ret_cust),21)` **rel_ret_cust 的实现。**

```

cust_offset = dr.GetData('rlv_cust_offset')
cust_est = dr.GetData('rlv_cust_id')
ret = dr.GetData("returns")

def Generate(di,alpha):
    for ii in xrange(len(alpha)):
        start, num = cust_offset[di-delay][ii]
        if (start >= 0 and num > 0):
            total_res = 0
            # off 应在 start 和 (start + num) 之间
            for off in xrange(start, start + num):
                # 计算 customers 的收益率
                if (cust_est[off] >= 0 and np.isnan(ret[di-delay][cust_est[off]])==False):
                    total_res = total_res + ret[di-delay][cust_est[off]]
            alpha[ii] = total_res

```

Example 2: 以下示例等同于表达式 `sum(log(rel_ret_comp),21)`

```

comp_offset = dr.GetData('rlv_comp_offset')
comp_est = dr.GetData('rlv_comp_id')
ret = dr.GetData("returns")

def Generate(di,alpha):
    for ii in xrange(len(alpha)):
        start, num = comp_offset[di-delay][ii]
        if (start >= 0 and num > 0):

```

```
for off in xrange(start, start + num):

local_res = 0

# 循环遍历 22 天以计算 sum

for j in range(22):

# 计算 rel_ret_comp 的 log

local_res = local_res + log(ret[di - delay - j][comp_est[off]])

total_res = total_res + local_res

alpha[ii] = total_res
```

Example 3: 以下 alpha 以不同方式计算公司竞争对手的月度收益。cap 的月度收益比 close 是更好的估计，因为它对股票拆分具有稳健性。该 alpha 的 Simulation 速度比上面的快得多，因为它没有额外的 for 循环。

```
comp_offset = dr.GetData('rlv_comp_offset')

comp_est = dr.GetData('rlv_comp_id')

vec_cap = dr.GetData("cap")

# close, open, ..., 其他 check fundamental/analysts 字段也可以在此处检查

def Generate(di,alpha):

for ii in xrange(len(alpha)):

start, num = comp_offset[di-delay][ii]

if (start >= 0 and num > 0):

total_res = 0

# off 应在 start 和 (start + num) 之间

for off in xrange(start, start + num):

# 计算 cap 的月度收益率

local_res = -1 + vec_cap[di - delay][comp_est[off]]/vec_cap[di - delay - 21][comp_est[off]]
```

高级预估数据

现在 WebSim 中已提供可用于在 Python 模式下回测 alphas 的高级预估数据。这是在支持表达式 alphas 的股票基础预估数据之外新增的内容。

- 有 29 个数据值（eps、sales 等）和 14 个字段（offset、estimate、ann_date 等）。
- 每个数据值都可使用全部 14 个字段，总计 406 个数据字段

数据名称	描述	范围
offset（偏移量）	其他字段的偏移量 (int[2])	NA
ann_date	公告日期 (int)	任意 Date <= Dates[dij] (YYYYMMDD 格式)
经纪商	经纪商 ID (int)	>=0
cons_date	一致预期日期 (int)	任意 Date <= ann_date
cons_inc	一致预期纳入 / 排除 (char)	'0', '1'
curr	货币 (int)	范围：[0,200)。每个整数对应一种货币。确切的货币代码可使用 currencylx 字段获取。示例：currencylx[curr [off]] 将返回 "USD"
est_dir	估计方向 (char)	U,D,F,N U-上调，D-下调，F-持平（无变化），N：新预测
估计值	估计值 (float)	任意 Float。其单位与 curr 相同（例如美国为 \$）
财政日期	财政期间结束日期 (int)	季度末：YYYY0331, YYYY0630, YYYY0930, YYYY1231
adj_date	调整日期 (int)	任意 Date <= ann_date
分析师	分析师 ID (int)	>=0
prev_est（先前预估）	先前预估值 (float)	任意 float。其单位与 curr 相同（例如美国为 \$）
rel_date	期间相对日期 (int)	通常为 [-15, +15]。大约等于 fisc_date - resr_date（以季度计）
resr_date	研究日期 (int)	任意接近 ann_date 的 Date

模拟

阿尔法

学习

数据

比赛（1次）

模拟	阿尔法
divps	每股股息
eps	每股收益
销售额	销售额
bvps	每股账面价值
capex	资本支出
cff	融资活动现金流
cfi	投资活动现金流
cfo	经营活动现金流
ebit	息税前利润
ebitda	息税折旧摊销前利润
epsa	调整后每股收益
epsr	已报告每股收益
自由现金流	自由现金流
每股自由现金流	每股自由现金流
营运资金资产比	调整后营运资金
营运资金	运营资金
grsinc	总收入
ntdebt（净债务）	净债务
ntprftadj（调整后净利润）	调整后净利润
ntprft（净利润）	净利润
ptp	税前收入
ptpr	已报告税前收入
rdexp	研发费用
销售、一般及行政费用	销售、一般及管理费用
股东权益	股东权益
TBVPS：每股有形账面价值	每股有形账面价值
Totasset：总资产	总资产
totgdwl：商誉总额	商誉总额

模板：

sales_est = 博士。GetData (“etv_sales_estimate”) ;用于获取销售额预测数据。

- 所有 etv_<DataName>_offset 字段都以 MATRIX 存储，并通过 [di][ii] 访问，返回一个元组 (start, num)。这里，start 是向量数据中的起始位置，而 num 是条目数量。

sales_offset = 博士。GetData ('etv_sales_offset') def Generate (di, alpha): for ii in xrange (alpha.size ()): start, num = sales_offset[di][ii] 用于获取销售额偏移数据，并在 Generate 中遍历 alpha.size ()。

- 非 offset 字段以 VECTOR 存储，并通过 <full_field_name>[off] 访问，其中 off 应位于 start 和 (start + num) 之间

sales_offset = 博士。GetData ('etv_sales_offset') sales_est = dr.GetData ('etv_sales_estimate') def Generate (di, alpha): for ii in xrange (len (alpha)): start, num = sales_offset[di][ii] 用于获取销售额偏移和销售额预测数据，并在 Generate 中遍历 len (alpha)。

if (起始 >= 0, 数字 > 0): total_sales_est = 0, 表示在 xrange 中 (起始, 起始 + 数字): total_sales_est = 6 total_sales_est + sales_est[关闭]

alpha[ii] = total_sales_est

注意：

1. 同一 DataName 的字段共享 offset。因此 etv_sales_offset 将为所有 etv_sales_<FieldName> 提供 offsets。
2. 这些数据字段范围仅适用于 “VALID” 数据。对于无效数据，可能会得到超出这些范围的值：例如 NAN、-1、-2147483648 等。此类值应由 alpha 处理并过滤掉。

上一集：更多数据集

下一篇：团队合作与重量共享